1. Problem Title & Source

Title: **Team Selection**

Source: **TCS NQT Previous Coding Questions**

2. Problem Statement (Clear Version)

A company is organizing a **team competition**.

There are **N participants**, and each participant has a **skill level** represented by an integer.

The organizer wants to form **teams of two participants**.

However, a team is considered **valid only if the difference between the skill levels of the two participants is exactly K**.

Your task is to determine **how many such valid teams can be formed**.

Important rules:

- Each participant can belong to **at most one team**.
- The order of pairing does not matter.
- Each pair must satisfy:

|skill[i] - skill[j]| = K

3. Examples

Example 1

Input

5

1 5 3 4 2

2

Output

3

Explanation

Possible pairs:

(1,3)

(3,5)

(2,4)

Each pair has difference:

2

Total valid teams:

3

Example 2

Input

4

8 12 16 4

4

Output

3

Explanation

Pairs

(4,8)

(8,12)

(12,16)

Each difference = **4**.

4. Constraints

1 ≤ N ≤ 1000

0 ≤ skill[i] ≤ 10^5

0 ≤ K ≤ 10^5

Observations

- Checking all pairs directly would take **$O(N^2)$**.
- We can optimize using a **HashSet**.

5. Core Concept (Pattern)

This problem uses:

Hashing

Pair difference detection

6. Thought Process

Brute Force Idea

Check all possible pairs.

```
for i
   for j
      if abs(skill[i] - skill[j]) == K
         count++
```

Time complexity

$O(N^2)$

Too slow for large N.

Optimized Idea

Use a **HashSet** to store skill values.

For each skill value x:

Check whether

x + K

exists in the set.

If yes → valid pair.

7. Visualization

Example

skills = [1,5,3,4,2]

K = 2

Set

{1,2,3,4,5}

Check

Skill	Skill+K	Exists?
1	3	Yes
5	7	No
3	5	Yes
4	6	No
2	4	Yes

Count

3

8. Pseudocode

```
read N
read array skills
read K

create set from skills

count = 0

for each skill

   if skill + K exists in set
      count++
```

print count

9. Code Implementation

Python

```
n = int(input())
arr = list(map(int,input().split()))
k = int(input())

s = set(arr)

count = 0

for x in arr:

    if x + k in s:
        count += 1

print(count)
```

Java

```
import java.util.*;

class Main {

    public static void main(String[] args){

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[] arr = new int[n];

        for(int i=0;i<n;i++)
            arr[i] = sc.nextInt();

        int k = sc.nextInt();
```

```
        Set<Integer> set = new HashSet<>();

        for(int x : arr)
            set.add(x);

        int count = 0;

        for(int x : arr){

            if(set.contains(x + k))
                count++;
        }

        System.out.println(count);
    }
}
```

10. Time & Space Complexity

Time Complexity

O(N)

HashSet lookups are **O(1)**.

Space Complexity

O(N)

For storing elements in the set.

11. Common Mistakes

1. Using nested loops unnecessarily
2. Counting duplicate pairs
3. Confusing difference direction
4. Forgetting absolute difference condition

12. Detailed Dry Run

Input

skills = [1,5,3,4,2]

K = 2

Set

{1,2,3,4,5}

Check

Skill	Skill+K	Pair
1	3	(1,3)
3	5	(3,5)
2	4	(2,4)

Result

3

13. Real-Life Applications

Pair difference problems appear in:

- Team formation
- Matching problems
- Data clustering
- Skill-based grouping

14. Common Traps

1. Using absolute difference incorrectly
2. Double counting pairs
3. Not handling duplicates
4. Forgetting hash structure

15. Related Problems

Pair problems:

LC 532 K-diff Pairs in an Array

LC 167 Two Sum II

LC 1 Two Sum

16. Alternate Approaches

Approach	Complexity	Notes
Brute force	$O(N^2)$	slow
HashSet	$O(N)$	optimal

17. Why This Solution Works

By storing all skill values in a **HashSet**, we can quickly check if the required matching value exists.

This avoids checking every possible pair.

18. Variations / Follow-Up Questions

Possible interview variations:

1. Find pairs with **sum = K**
2. Find pairs with **difference ≤ K**
3. Count **unique pairs**
4. Return **actual team combinations**